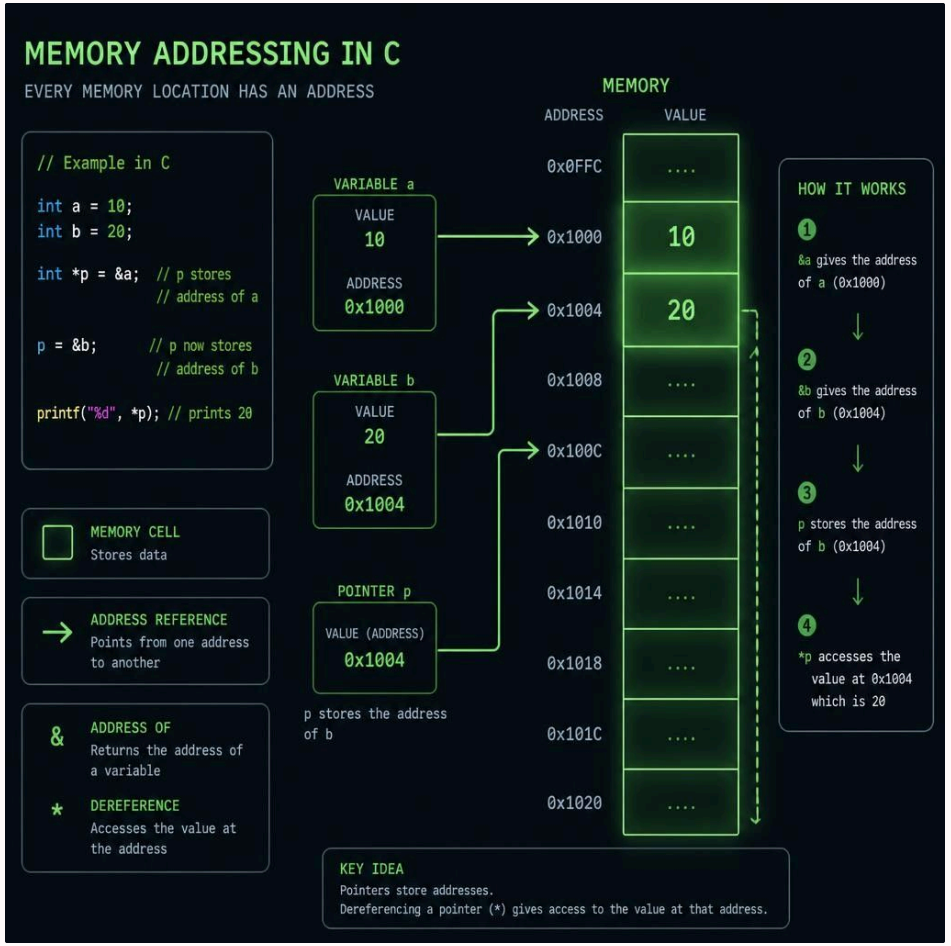
Puntatori a Puntatori in C

Doppio puntatore, array di puntatori, matrici e puntatori

LINGUAGGIO C | PUNTATORI | PUNTATORI A PUNTATORI

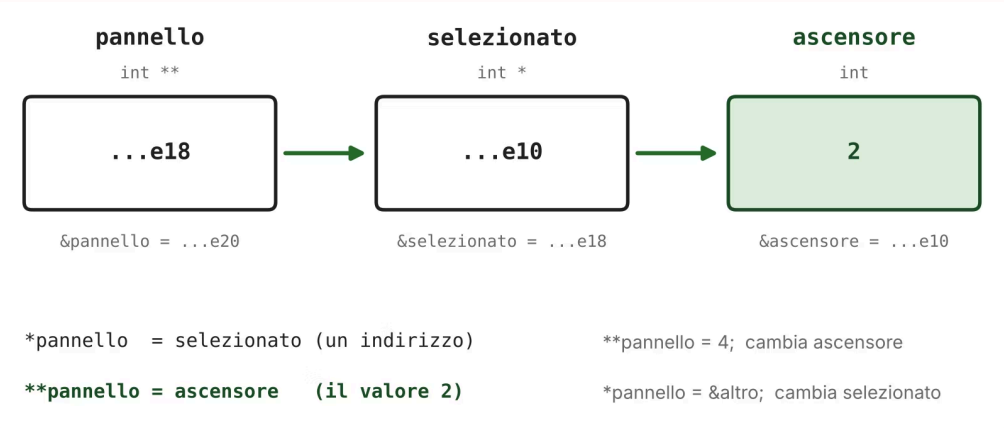


Il Puntatore a Puntatore

Anche un puntatore è una variabile e ha un indirizzo. Un puntatore a puntatore (doppio puntatore) contiene l'indirizzo di un altro puntatore.

```
int ascensore = 2;
int *selezionato = &ascensore;
int **pannello = &selezionato;

**pannello = 4;          /* ascensore vale 4 */
*pannello = &montacarichi;   /* selezionato cambia bersaglio */
```



pannello è l'indirizzo di selezionato; *pannello è il valore di selezionato (l'indirizzo di ascensore); **pannello è il valore di ascensore.

Modificare un Puntatore del Chiamante

```
void scegli_coda(int **coda, int *c1, int *c2, int *c3)
{
   *coda = c1;
   if (*c2 < **coda) {
      *coda = c2;
   }
   if (*c3 < **coda) {
      *coda = c3;
   }
}

scegli_coda(&mia_coda, &cassa1, &cassa2, &cassa3);
```

Il problema: una funzione deve far puntare un puntatore del main alla cassa del supermercato con la fila più corta.

La soluzione: come per modificare un int serve un int *, per modificare un int * serve un int **: si passa l'indirizzo del puntatore.

Con il parametro int *coda la funzione cambierebbe solo la propria copia del puntatore: nel main mia_coda punterebbe ancora alla cassa 1.

Il Puntatore che Avanza

```
int leggi_numero(const char **p, int *numero)
{
    int letto = 0;
    while (**p == ' ') {
        (*p)++;
    }
    *numero = 0;
    while (**p >= '0' && **p <= '9') {
        *numero = *numero * 10 + (**p - '0');
        (*p)++;
        letto = 1;
    }
    return letto;
}
```

Il problema: leggere uno dopo l'altro i numeri scritti in una riga di testo, come "12 7 130 45 9".

La soluzione: la funzione riceve l'indirizzo del puntatore di lettura (const char **) e lo sposta oltre il numero letto. Alla chiamata successiva la lettura riparte da lì.

Espressione	Significato
`*p`	il puntatore di lettura del chiamante
`**p`	il carattere da leggere
`(*p)++`	sposta in avanti il puntatore del chiamante

Le parentesi in (*p)++ sono necessarie: *p++ sposterebbe la copia locale p, non il puntatore del chiamante.

Esito con return, Puntatore in Uscita

Il problema: una ricerca deve dire se ha trovato l'elemento e, se sì, permettere di modificarlo.

La soluzione: l'esito (1 o 0) esce con return, il puntatore all'elemento trovato esce da un parametro di tipo int **.

```
int cerca(int *v, int n, int valore, int **trovato);

if (cerca(posti, POSTI, 0, &posto)) {   /* primo posto libero */
    *posto = tessera;
    printf("Posto %d", (int) (posto - posti) + 1);
}
```

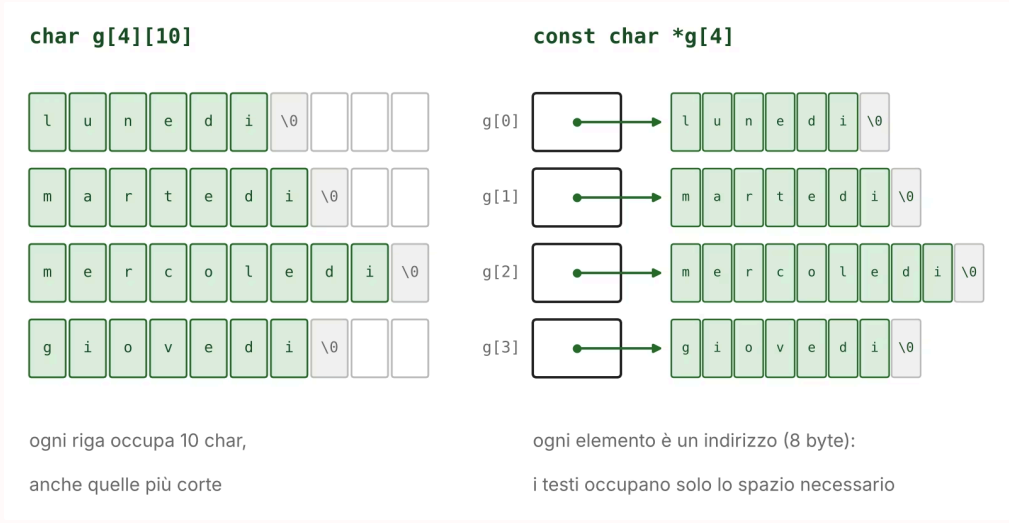
Se non trova, cerca scrive NULL in *trovato e restituisce 0: il chiamante dereferenzia posto solo dopo un esito positivo.

Array di Puntatori a Stringhe

Un elenco di testi può essere un array di puntatori: ogni elemento è un const char * che punta a un letterale lungo quanto serve.

```
const char *brevi[7] = {"lun", "mar", "mer", "gio", "ven", "sab", "dom"};

printf("%s", brevi[i]);     /* la stringa i-esima */
brevi[i][0]                 /* il suo primo carattere */
brevi[6] = "---";           /* cambia il puntatore, non il testo */
```



Una matrice char[7][10] riserva 10 caratteri per ogni riga, anche per "sab"; l'array di puntatori occupa 7 indirizzi più i testi, ciascuno della sua lunghezza.

Puntatori a Variabili e ad Array

Un array di puntatori raccoglie in un elenco variabili o array che nel programma sono separati, per elaborarli con un ciclo.

```
int *sensori[3] = {&aula, &laboratorio, &palestra};
*sensori[i] = *sensori[i] + 10;      /* modifica la variabile i-esima */

int *gruppi[4] = {rossi, blu, verdi, gialli};
int studenti[4] = {3, 5, 4, 2};
media(gruppi[g], studenti[g]);       /* array del gruppo g */
gruppi[g][i]                         /* voto i del gruppo g */
```

Le variabili e gli array restano dove sono: l'array contiene solo i loro indirizzi. Per gruppi di dimensioni diverse serve un secondo array con le lunghezze.

Riordinare Scambiando Puntatori

Il problema: cambiare l'ordine di un elenco di nomi senza copiare i caratteri.

La soluzione: si scambiano due indirizzi dell'array di puntatori; i testi restano dove sono. Per scambiare due const char * la funzione riceve i loro indirizzi: const char **.

```
void scambia(const char **a, const char **b)
{
    const char *temp = *a;

    *a = *b;
    *b = temp;
}

scambia(&ordine[0], &ordine[2]);
```

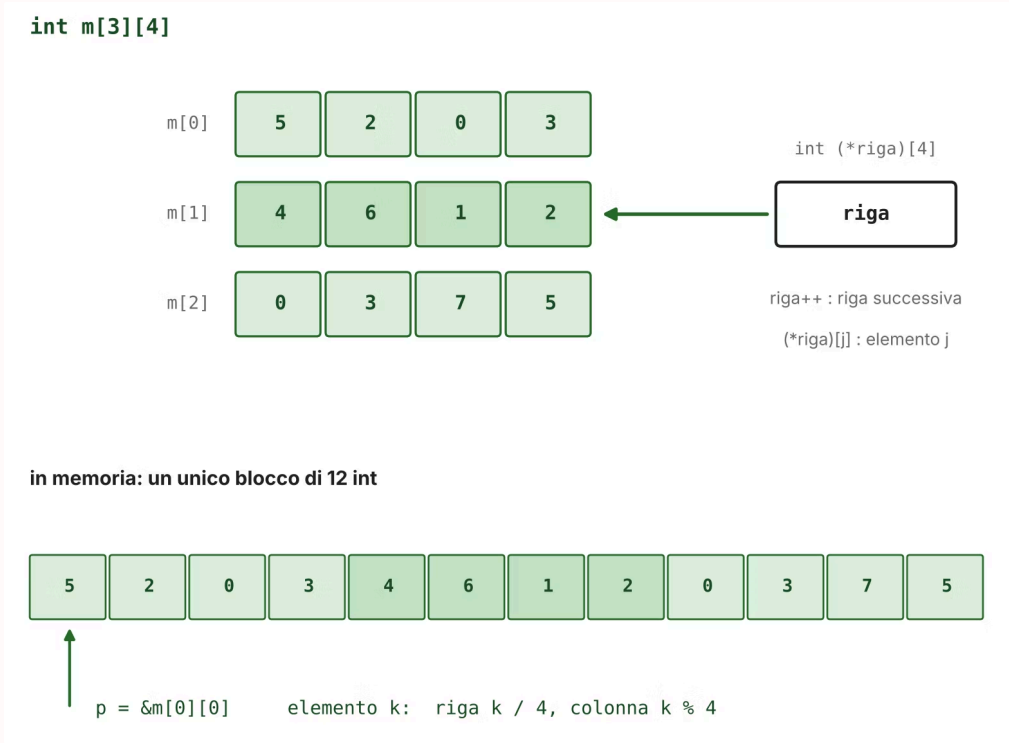
Anche le funzioni si possono raccogliere in un array: void (*comandi[4])(int *, int *, int *) = {avanti, indietro, destra, sinistra}; e poi comandi[k](&x, &y, &dir).

La Matrice in Memoria

Una matrice int m[R][C] è un array di R righe, ciascuna un array di C int. In memoria le righe sono consecutive: tutta la riga 0, poi tutta la riga 1, e così via.

Espressione	Significato
`m[i]`	la riga i: un array di C int
`m + i`	indirizzo della riga i: avanza di una riga intera
`*(*(m + i) + j)`	l'elemento m[i][j]
`&m[0][0] + i * C + j`	lo stesso elemento contato dall'inizio

Gli indirizzi sono numeri esadecimali, come li stampa %p: tra un elemento e il successivo c'è sizeof(int) = 4 byte, tra una riga e la successiva C × 4 byte.



Tre Modi di Scorrere una Matrice

```
int (*riga)[MESI];

for (riga = vendite; riga < vendite + NEGOZI; riga++) {
    totale = totale_riga(riga);      /* (*riga)[j] è l'elemento j */
}

media(&serra[0][0], FILE_SERRA * SETTORI);   /* tutta la serra */
media(serra[f], SETTORI);                    /* una fila */
/* posizione k = p - &m[0][0]:  riga k / C, colonna k % C */
```

Tecnica	Codice	Quando usarla
Puntatore a riga	`int (*riga)[C]; riga++`	elaborare riga per riga
Una riga come array	`f(m[i], C)`	riusare funzioni su int *
Blocco unico	`p = &m[0][0]; p < &m[0][0] + R * C`	la riga non conta

int (*riga)[C] è un puntatore a una riga; int *riga[C] è un array di C puntatori: le parentesi cambiano tutto.

Righe di Lunghezza Diversa

Il problema: le distanze tra cinque paesi sono simmetriche, quindi basta la metà della tabella sotto la diagonale; la riga i ha i elementi.

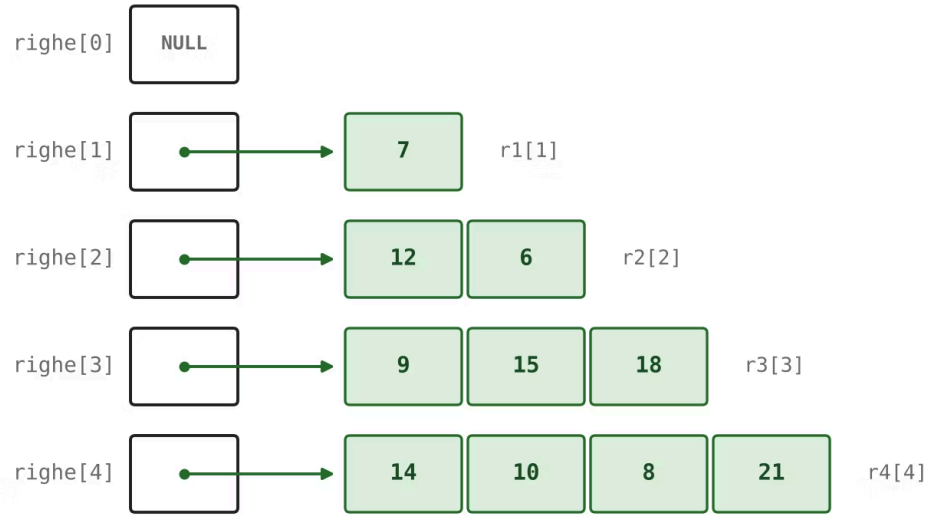
La soluzione: ogni riga è un array a sé e un array di puntatori le raccoglie.

```
int r1[1] = {7};
int r2[2] = {12, 6};
int r3[3] = {9, 15, 18};
int r4[4] = {14, 10, 8, 21};
const int *righe[5] = {NULL, r1, r2, r3, r4};

righe[3][1]      /* distanza tra il paese 3 e il paese 1 */
```

Con un array di puntatori alle righe, riordinare (ad esempio una classifica) significa scambiare puntatori: le righe della matrice non si spostano.

```
const int *righe[5] = {NULL, r1, r2, r3, r4};
```



Riga i: distanze dal paese i ai paesi 0 .. i-1. righe[3][1] = 15 km.

Errori Comuni da Evitare

1

Passare int * a una funzione che deve cambiare un puntatore del chiamante: serve int **.

2

Scrivere *p++ invece di (*p)++ per far avanzare il puntatore del chiamante.

3

Confondere *pp (il puntatore intermedio) con **pp (la variabile finale).

4

Dereferenziare il puntatore in uscita senza controllare l'esito o NULL.

5

Modificare il testo di un letterale puntato da un array di const char *.

6

Scrivere int *riga[C] (array di puntatori) invece di int (*riga)[C].

7

Usare riga[j] invece di (*riga)[j] con un puntatore a riga.

8

Ricavare riga e colonna dividendo per il numero di righe invece che per la lunghezza di una riga.

Riepilogo Concetti

Operazione	Tecnica	Note
Puntatore a puntatore	`int **pp = &p`	*pp è p, **pp è la variabile
Cambiare un puntatore del chiamante	`f(&p)` con parametro `int **`	*coda = c2
Lettura che avanza	`const char **p, (*p)++`	riprende da dove si è fermata
Esito e puntatore	`return 0/1, int **trovato`	NULL se non trovato
Elenco di testi	`const char *v[N]`	testi di lunghezza diversa
Elenco di variabili	`int *v[N] = {&a, &b}`	*v[i] è la variabile
Riordinare	scambiare puntatori	i dati non si spostano
Tabella di funzioni	`void (*f[N])(...)`	f[k](...)
Puntatore a riga	`int (*riga)[C]`	riga++ avanza di una riga
Matrice come blocco	`&m[0][0], R * C`	riga k / C, colonna k % C
Righe di lunghezza diversa	`const int *righe[N]`	righe[i][j]

Esempi Pratici: Doppio Puntatore

Concetti base — tre file sul puntatore a puntatore.

1 – 010-puntatore-a-puntatore.c

Problema: ascensore e montacarichi selezionati da un pannello.

Tecnica: int **pannello, *pannello e **pannello.

2 – 020-modificare-un-puntatore-del-chiamante.c

Problema: scegliere la cassa con la fila più corta al supermercato.

Tecnica: scegli_coda(&mia_coda, ...) con parametro int **.

3 – 030-puntatore-che-avanza.c

Problema: punteggi delle squadre di un quiz scritti in una riga.

Tecnica: leggi_numero(&p, &n) con const char ** e (*p)++.

Esempi Pratici: Problemi con il Doppio Puntatore

040 – Turno di guardia

Sportello di tutoraggio: passa_turno(&turno, ore, n) sposta il docente di turno.

050 – Calcolatrice delle spese

Espressioni di entrate e uscite lette con un solo puntatore che avanza; posizione dell'errore.

060 – Coppia più vicina

Fotofinish della campestre: i due tempi più vicini restituiti con const int **.

070 – Lettore di comandi

Domotica dell'aula: SET luce 80; GET luce letti da funzioni che fanno avanzare la lettura.

080 – Ricerca con esito

Rastrelliera delle biciclette: esito con return, posto trovato in un int **.

Esempi Pratici: Array di Puntatori

Concetti base — tre file sugli array di puntatori.

1 – 010-array-di-stringhe-costanti.c

Problema: giorni di apertura della biblioteca.

Tecnica: const char *giorni[7] a confronto con char[7][10].

2 – 020-array-di-puntatori-a-variabili.c

Problema: temperature di aula, laboratorio e palestra.

Tecnica: int *sensori[3] = {&aula, ...} e *sensori[i].

3 – 030-scambiare-puntatori.c

Problema: ordine degli interventi in assemblea di classe.

Tecnica: scambia(&ordine[0], &ordine[2]) con const char **.

Esempi Pratici: Problemi con Array di Puntatori

040 – Menu del giorno

Mensa scolastica: piatti con nomi di lunghezza molto diversa.

050 – Classifica per puntatori

Gara di orientamento: la giuria corregge la classifica scambiando puntatori a nomi e tempi.

060 – Messaggi di errore

Stampante 3D: il codice dell'errore è l'indice della descrizione.

070 – Tabella di comandi

Robot del laboratorio: un array di puntatori a funzione al posto di una catena di if.

080 – Gruppi di lavoro

Voti di gruppi con un numero diverso di studenti: int *gruppi[4].

Esempi Pratici: Matrici e Puntatori

Concetti base — tre file sulla matrice vista con i puntatori.

1 – 010-matrice-in-memoria.c

Problema: prenotazioni dei banchi di un'aula studio.

Tecnica: indirizzi consecutivi, sizeof, *(*(m + i) + j).

2 – 020-puntatore-a-riga.c

Problema: vendite mensili di quattro cartolerie.

Tecnica: int (*riga)[MESI] e riga++.

3 – 030-matrice-come-blocco-unico.c

Problema: minuti di occupazione del parcheggio della scuola.

Tecnica: &m[0][0], un solo ciclo, riga k / C e colonna k % C.

Esempi Pratici: Problemi su Matrici e Puntatori

040 – Orario settimanale

La riga del giorno passata come const int *, la matrice come const int (*) [ORE].

050 – Statistiche della serra

Le stesse funzioni su tutta la serra (blocco unico) e su una fila (una riga).

060 – Triangolo delle distanze

Pulmino scolastico: tabella triangolare con righe di lunghezza diversa.

070 – Scambio di righe

Torneo di calcetto: classifica come array di puntatori alle righe, sorpassi con scambi.

080 – Immagine in scala di grigi

Capovolgimento con due puntatori a riga, negativo come blocco unico.